

1. Proponga un algoritmo para evaluar la función de Airy del primer tipo $\text{Ai}(x) = \frac{1}{\pi} \int_0^\infty \cos\left(\frac{t^3}{3} + xt\right) dt$ rápidamente para valores de $x \in [-10, 20]$.
2. ¿Cuándo un algoritmo de integración numérica entrega resultados exactos?
3. ¿Qué algoritmo utilizaría para integrar en el intervalo $[-1, 1]$ exactamente las siguientes funciones? Y también obtenga el valor de la integral definida correspondiente. La idea es utilizar el algoritmo más simple posible en cada caso, ya que, en principio, los “siguientes” algoritmos también encontrarán la integral de forma exacta.
 - $f(x) = 2$.
 - $f(x) = 2x$.
 - $f(x) = 2x + 1$.
 - $f(x) = ax + b$, donde a y b son valores escalares reales pero desconocidos.
 - $f(x) = ax^2 + bx$, donde a y b son valores escalares reales pero desconocidos.
 - $f(x) = ax^3 + bx^2 + c$, donde a , b y c son valores escalares reales pero desconocidos.
 - $f(x) = \sin(x)$.
 - $f(x) = \sum_{i=0}^5 a_i x^i$, para $a_i \in \mathbb{R}$.
 - $f(x) = \frac{\frac{1/6}{x-0} - \frac{3/2}{x-2}}{\frac{1/6}{x-0} - \frac{1/2}{x-2} + \frac{1/3}{x-3}}$.
 - $f(x) = \sum_{i=0}^{n-1} a_i x^i = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}$.
4. Considere que usted acaba de interpolar la función exponencial, es decir $\exp(x)$, con n puntos de Chebyshev y utilizando interpolación Baricéntrica en el intervalo $[-10, 5]$. Por simplicidad denotaremos el interpolador como $p(x)$.
 - ¿Qué algoritmo de integración numérica utilizaría para obtener $\int_{-1}^1 p(x) dx$ con un error absoluto menor a 10^{-10} ?
 - ¿Cuántos puntos utilizaría con la cuadratura Gaussiana $\int_{-1}^1 p(x) dx$ con un error absoluto menor a 10^{-10} ?
5. Considere que usted interpoló con n puntos de Chebyshev las funciones $\sin(x)$ y $\cos(x)$ con los polinomios $p_s(x)$ y $p_c(x)$ para construir una aproximación numérica de la función $f(x) = \sin(\alpha \cos(x))$, es decir $f(x) \approx f_a(x) = p_s(\alpha p_c(x))$, para un valor real de α conocido. ¿Cuántos puntos de integración utilizando la cuadratura Gaussiana aseguran obtener el valor **exacto** de $\int_{-1}^1 f_a(x) dx$?

6. Uno de los operadores más famosos en Matemática es la derivada de una función, es decir $\frac{d}{dx}f(x)$. Más aún, si es que la función en estudio lo permite, uno puede obtener la derivada de la derivada, o mejor conocida como la segunda derivada $\frac{d^2}{dx^2}f(x)$, y así sucesivamente. En general podríamos obtener la n -ésima derivada, donde $n \in \mathbb{N}$. Pero uno se podría preguntar lo siguiente: ¿Por qué n debe ser un número natural? ¿Podría n ser un número real? ¿Podría $n \in]0, 1[$? Es decir, ¿se podría obtener una derivada intermedia, o mejor conocida como, una derivada fraccionaria? La respuesta es sí, y se presentará una alternativa conocida por el nombre Derivada fraccionaria izquierda de Riemann-Liouville y se define de la siguiente forma:

$$\frac{d^\alpha}{dx^\alpha}f(x) = \frac{1}{\Gamma(\lceil\alpha\rceil - \alpha)} \frac{d^{\lceil\alpha\rceil}}{dx^{\lceil\alpha\rceil}} \int_0^x (x-y)^{\lceil\alpha\rceil-\alpha-1} f(y) dy,$$

donde $\Gamma(x)$ corresponde a la función Gamma, y $\lceil\alpha\rceil$ corresponde a la parte entera superior de α . En particular, si $\alpha \in]0, 1[$ se puede simplificar de la siguiente forma:

$$(1) \quad \frac{d^\alpha}{dx^\alpha}f(x) = \frac{1}{\Gamma(1-\alpha)} \frac{d}{dx} \int_0^x (x-y)^{-\alpha} f(y) dy,$$

Por simplicidad uno puede definir $g(x) = \int_0^x (x-y)^{-\alpha} f(y) dy$ y obtener,

$$\frac{d^\alpha}{dx^\alpha}f(x) = \frac{1}{\Gamma(1-\alpha)} \frac{d}{dx}g(x).$$

Ahora, para obtener una aproximación numérica uno puede construir un algoritmo con los siguientes pasos:

- Aplique forward difference para aproximar $\frac{d}{dx}g(x)$ considerando h como la variación en x . Su explicación debe desarrollarse en LaTeX.
- Explique teóricamente como puede aproximar numéricamente la derivada fraccionarias definida en la ecuación (1) considerando la aproximación de diferencias finitas obtenida en la pregunta anterior. Es decir, debe explicar cómo obtendrá las integrales obtenidas para una función $f(x)$ arbitraria. Por simplicidad considere que se necesita evaluar la derivada fraccionaria para $0 < x$. Su implementación debe utilizar la cuadratura Gaussiana, es decir debe recibir como input los nodos y pesos de la cuadratura Gaussiana en el intervalo $[-1, 1]$.
- Implemente su aproximación.

```
'''
INPUT:
x_hat    : (double)    Punto donde se evaluará la derivada fraccionaria.
f        : (callable)  Implementación de la función $f(x)$.
alpha    : (double)    Vector de coeficientes w
h        : (double)    Variación en x para calcular forward difference.
x_gauss  : (ndarray)   Nodos de la cuadratura Gaussiana en [-1,1].
w_gauss  : (ndarray)   Pesos de la cuadratura Gaussiana en [-1,1].

OUTPUT:
out      : (double)    Derivada fraccionaria de f(x) evaluada en x_hat
'''

def fractional_derivative(x_hat,f,alpha,h,x_gauss,w_gauss):
    # Your own code.
    out = 0 # Remover esta línea si usted implementa esta función.
    return out
```

- Obtenga el valor de la derivada fraccionaria con lo siguientes parámetros:

```
x_hat = 0.5
f      = lambda x: x
alpha = 0.5
h      = 0.01
x_gauss, w_gauss = gaussian_nodes_and_weights(50, -1, 1)
```